

CSCE 633 MACHINE LEARNING · MODULE 07 · LECTURE NOTES

Kernels and Margins

Infinite features without computing any of them.

MODULE OUTCOMES

- Explain the maximum margin principle.
- Explain the kernel trick and the representer theorem.
- Choose a kernel and explain its hyperparameters.
- Explain why SVMs lost ground and where they still fit.
- Connect kernels to the other methods in the course.

1 The margin

MAXIMISE THE DISTANCE TO THE NEAREST POINT

If the data is linearly separable, infinitely many hyperplanes separate it, and they are not equally good. Logistic regression selects one by maximising likelihood; the SVM selects one by a purely geometric criterion.

Choose the hyperplane that maximises the distance to the nearest training point on either side — the margin. This is a well-posed optimisation with a unique solution.

The intuition is that a wide margin is a confident decision, and leaves the greatest room for a new point drawn from the same distribution to fall on the correct side. There are generalisation bounds that formalise this, and they are loose, but the intuition survives.

Only the closest points affect the solution. These are the *support vectors*, and moving or deleting any other training point changes nothing at all — which gives the method its name, makes the model compact (prediction needs only the support vectors), and is its most distinctive property. It also means the fit is robust to most of the data and exquisitely sensitive to a few points, which cuts both ways.

$$\min \quad \frac{1}{2} \|w\|^2 + C \sum \xi_i \quad \text{s.t.} \quad y_i (w^T x_i + b) \geq 1 - \xi_i$$

Real data is not separable, so the hard margin is replaced by a *soft* one: each point should be on the correct side by a margin of at least 1, and a slack variable ξ pays for the shortfall. **Large C means violations are expensive, so the margin is narrow and the fit tight; small C means a wide margin and more violations tolerated.** C is Module 04's regularisation dial in different notation — it is the inverse of λ , and noticing that the two are the same knob prevents treating SVM tuning as a separate skill.

2 The kernel trick

THE OBSERVATION

the SVM's solution depends on the data ONLY through inner products $x_i \cdot x_j$ -- never through any x_i alone.

SO to fit in a richer feature space $\phi(x)$, you need only $\phi(x_i) \cdot \phi(x_j)$ and NOT $\phi(x)$ itself.

A KERNEL computes that inner product directly:

$$K(x, z) = \phi(x) \cdot \phi(z)$$

POLYNOMIAL $K = (x \cdot z + 1)^d$

ϕ expands to all monomials up to degree d .

$d=5$ with 100 features is ~96 MILLION terms.

K costs one dot product and one power.

RBF / GAUSSIAN $K = \exp(-\gamma \|x - z\|^2)$

ϕ is INFINITE-dimensional -- it cannot be written down.

K is three operations.

You fit a linear model in a space you never construct.

THE REPRESENTER THEOREM IS WHY THIS IS LEGITIMATE

For a broad class of regularised problems, the optimal solution can be written as a weighted sum of kernel evaluations at the training points — $f(x) = \sum \alpha_i K(x_i, x)$. This is a theorem, not a convenient restriction.

So the solution lives in a space whose dimension is n , the number of training points, regardless of how high-dimensional the feature space ϕ maps into.

Infinite features, n parameters. This is why an infinite-dimensional model does not automatically overfit: the effective capacity is bounded by the sample size and the regularisation, not by the nominal dimension — which is the same distinction Module 03 §3 drew between parameter count and effective capacity.

And it is why the computational cost scales with n rather than with the feature dimension. That is the method's great strength on wide data — ten thousand features cost nothing extra — **and its fatal weakness on large data**, since the kernel matrix alone is $n \times n$ (§3).

Kernel	Form	Character
Linear	$x \cdot z$	An ordinary linear model. Try it first — on wide data it frequently wins, and it is far faster.
RBF (Gaussian)	$\exp(-\gamma \ x - z\ ^2)$	The default. γ sets how local the influence of each point is: large γ means a narrow bump and a wiggly boundary (high variance), small γ approaches linear.

Kernel	Form	Character
Polynomial	$(x \cdot z + 1)^d$	Explicit interactions up to degree d. Numerically awkward — large d produces very large or very small values and the optimisation suffers.
Sigmoid	$\tanh(\kappa x \cdot z + c)$	Not positive semidefinite for all parameters , so it is not always a valid kernel. Rarely used, and historically motivated by an analogy to neural networks.
String, graph, and custom kernels	Domain-specific similarity measures.	Where kernel methods still genuinely shine — you can define a similarity between objects that have no natural vector representation at all, which is a capability nothing else offers so directly.

γ and C **must be tuned together** — they interact strongly, since both control effective capacity by different routes, and tuning one at a time finds a poor optimum. **This is one of the few places where a grid over two parameters is genuinely defensible** (Module 04 §3), because the interaction is exactly what you need to see.

3 Where SVMs stand now

WHY SVMS LOST GROUND

Training is roughly $O(n^2)$ to $O(n^3)$ — the kernel matrix alone has n^2 entries — **and prediction requires a kernel evaluation against every support vector**, of which there may be thousands. **Past roughly 100,000 samples this is impractical**, and approximations (Nyström, random features) give up much of the advantage.

Gradient-boosted trees usually match or beat them on tabular data (Module 06 §3), with far less tuning, no feature scaling requirement, and native handling of categorical and missing values — all of which the SVM needs engineered.

And neural networks took the structured domains — images, audio, text — where kernel methods had been briefly competitive, by learning the representation rather than specifying it.

They remain excellent on small, wide, clean data: a few thousand samples and many features, which describes a great deal of bioinformatics and chemometrics. **And they remain the natural choice wherever a custom kernel encodes real domain structure** — comparing sequences, molecules, or graphs — because defining a similarity is sometimes far easier than defining a feature vector.

WHAT KERNELS TAUGHT THE REST OF THE FIELD

The margin idea became a loss function. Hinge loss and its relatives appear throughout deep learning, in metric learning, contrastive objectives, and ranking — the geometry outlived the algorithm.

The representer theorem explains how an n-sample method can work in a very high-dimensional space at all, which is precisely the question deep learning still wrestles with (Module 03 §4). The kernel answer — effective capacity is bounded by samples and regularisation, not by dimension — was the first clean statement of it.

And 'make the problem linear in a transformed space' is the entire strategy of representation learning. A deep network's final layer is a linear classifier operating on learned features (CSCE 636 Module 05) — structurally the same arrangement as a kernel machine.

So the difference is where the features come from: a kernel specifies them in advance, and a network learns them from the data. That one sentence is most of the history of the field between about 1995 and 2015, and it explains both why kernels won when data was scarce and why networks won when it was not — learning a representation requires enough examples to learn it from.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Stanford CS229 — support vector machines notes (free)	The derivation of §1 and §2 , including the dual formulation that makes the kernel substitution visible.
Hastie et al. — The Elements of Statistical Learning, chapter 12 (free PDF)	SVMs with the regularisation framing, which makes the C - λ connection of §1 explicit.
Schölkopf & Smola — Learning with Kernels	The reference for the representer theorem and for constructing custom kernels (§2). Library copy.
scikit-learn — SVM user guide and the kernel approximation module (free)	Practical tuning of C and γ , and the Nyström and random feature approximations that address §3's scaling problem.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Fit a linear SVM on separable data and **identify the support vectors**. Move a non-support point and confirm nothing changes.
- 2 Sweep C and plot the margin width and the violation count.
- 3 **Verify that C is the inverse of λ** by fitting an equivalent regularised linear model.
- 4 **Compute a degree-3 polynomial kernel explicitly** and via the kernel function. Compare the results and the cost.
- 5 Count how many explicit features a degree-5 polynomial kernel would need on your dataset.
- 6 **Fit an RBF SVM and sweep γ** , visualising the decision boundary at each value.
- 7 **Tune γ and C jointly on a grid** and plot the validation surface. Explain its shape.
- 8 Measure training time against sample size from 1,000 to 100,000 and confirm the superlinear growth.
- 9 **Compare an SVM against gradient-boosted trees** on a wide, small dataset and on a tall one. Report both.
- 10 Define a custom kernel for a non-vector object (strings or sets) and verify it is positive semidefinite.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 What does the SVM maximise, and which points affect the solution?
- 2 What does C control, and what is it the inverse of?
- 3 State the kernel trick and say what makes it possible.
- 4 Give two kernels and say what their feature spaces are.

- 5 State the representer theorem and its two consequences.
- 6 Why does SVM cost scale with n rather than d , and when is that good?
- 7 Give three reasons SVMs lost ground and two places they still fit.
- 8 What is the relationship between a kernel machine and a deep network?